



# Pixel

STREAMING DATABASE SYSTEM



By: Abdulaziz Shafae

# 1. Project Overview

Pixel is a database system designed for a simple movie and series streaming platform. The system stores information about movies, TV series, users, favorite lists, and viewing history.

Users can subscribe to the platform, add movies or series to their favorite list, and track the content they are currently watching or have already finished.

The database was created using MySQL and uses relationships between tables through Primary Keys and Foreign Keys.

## 2. Database Tables

The Pixel database contains five main tables:

### Movies

Stores information about the movies available on the platform.

Attributes:

- id – Unique movie ID
- title – Movie title
- subtitle – Movie subtitle
- length – Movie duration in minutes
- type – Movie genre

### Series

Stores information about the series available on the platform.

Attributes:

- id – Unique series ID
- title – Series title
- subtitle – Series subtitle
- episodes – Number of episodes
- type – Series genre

## Users

Stores information about registered users.

Attributes:

- id – Unique user ID
- username – User's username
- password – User's password
- email – User's email address
- subscribe – Indicates whether the user is subscribed

## FivList

Stores movies and series that users add to their list.

Each item can have one of three statuses: new, watching, or finish.

Attributes:

- id – Unique list record ID
- status – Current status of the content
- user\_id – User who added the content
- movie\_id – Movie reference, if the item is a movie
- series\_id – Series reference, if the item is a series

## History

Stores the user's viewing history.

The status indicates whether the user is currently watching the content or has finished it.

Attributes:

- id – Unique history record ID
- status – Viewing status
- user\_id – User reference
- movie\_id – Movie reference, if applicable
- series\_id – Series reference, if applicable

### 3. Database Relationships

The database uses one-to-many relationships.

A user can have multiple items in their favorite list and multiple records in their viewing history. A movie or series can also appear in the lists and histories of multiple users.

#### Database Diagram

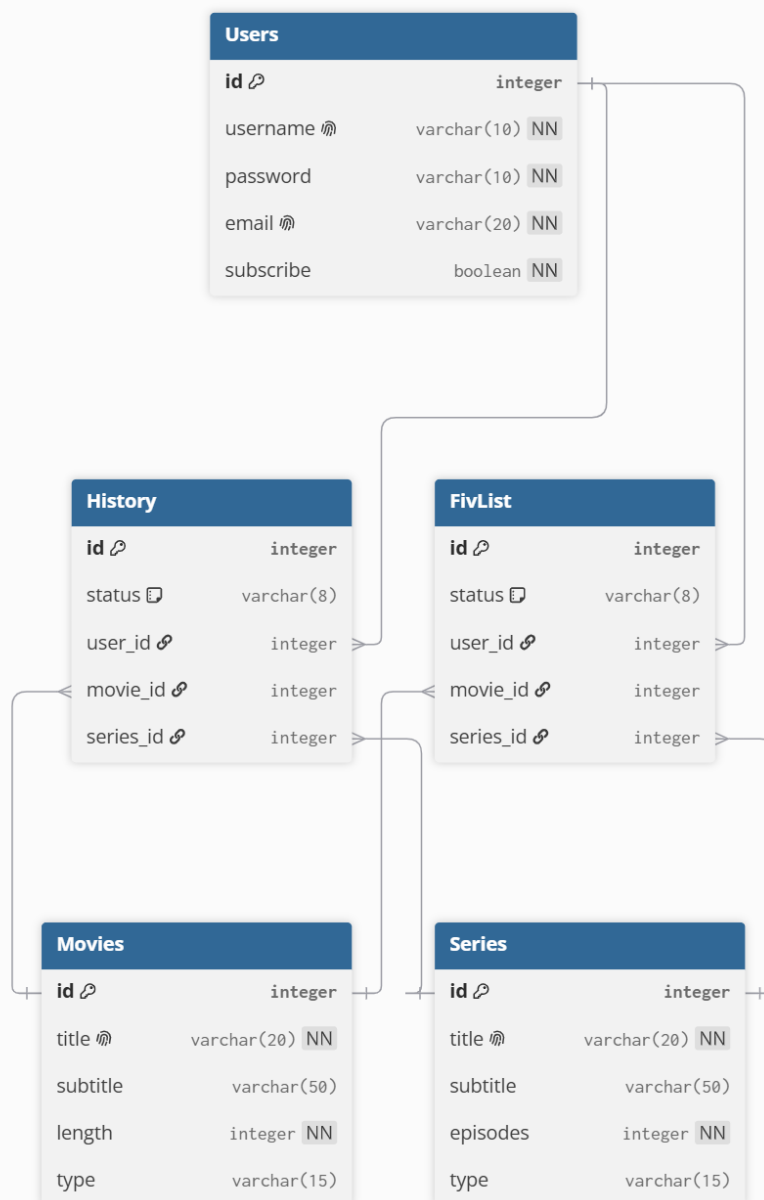


Figure 1: Pixel Database Schema

## 4. Database Constraints

Several SQL constraints are used to maintain the accuracy and integrity of the data.

PRIMARY KEY – Uniquely identifies each record.

AUTO\_INCREMENT – Automatically generates IDs for new records.

NOT NULL – Prevents required attributes from being empty.

UNIQUE – Prevents duplicate values such as usernames, emails, and titles.

CHECK – Restricts status values to the allowed options.

FOREIGN KEY – Creates relationships between the database tables.

## 5. SQL Implementation

The following section contains the SQL code used to create the Pixel database, create its tables, define the relationships, and insert sample data.

SQL Code

```
CREATE DATABASE Pixel;
```

```
CREATE TABLE Movies(  
  id int primary key auto_increment,  
  title varchar(20) unique not null ,  
  subtitle varchar(50),  
  length int not null,  
  type varchar(15)  
);
```

```
INSERT INTO Movies (title, subtitle, length, type) VALUES  
( 'Up', 'Adventure Is Out There', 96, 'Animation'),  
( 'Interstellar', 'Mankind Was Born on Earth', 169, 'Sci-Fi'),  
( 'Inception', 'The Dream Is Real', 148, 'Sci-Fi'),  
( 'Titanic', 'Nothing on Earth Could Come Between Them', 194, 'Romance'),  
( 'The Dark Knight', 'Why So Serious?', 152, 'Action');
```

```
CREATE TABLE Series(  
    id int primary key auto_increment,  
    title varchar(20) unique not null ,  
    subtitle varchar(50),  
    episodes int not null,  
    type varchar(15)  
);
```

```
INSERT INTO Series (title, subtitle, episodes, type) VALUES  
( 'Breaking Bad', 'Crime and Consequences', 62, 'Drama'),  
( 'Friends', 'Life in New York', 236, 'Comedy'),  
( 'Dark', 'Everything Is Connected', 26, 'Sci-Fi'),  
( 'Sherlock', 'The Consulting Detective', 13, 'Mystery'),  
( 'The Office', 'Life at Dunder Mifflin', 201, 'Comedy');
```

```
CREATE TABLE Users(  
    id int primary key auto_increment,  
    username varchar(10) unique not null,  
    password varchar(10) not null,  
    email varchar(20) unique not null,  
    subscribe boolean not null  
);
```

```
CREATE TABLE FivList(  
    id int primary key auto_increment,  
    status varchar(8) check ( status= 'new' or status= 'watching' or status= 'finish' ),  
    user_id int,  
    movie_id INT,  
    series_id INT,  
  
    FOREIGN KEY (movie_id) references Movies(id),  
    FOREIGN KEY (series_id) references Series(id),  
    FOREIGN KEY (user_id) references Users (id)  
);
```

```
CREATE TABLE History(  
    id int primary key auto_increment,  
    status varchar(8) check ( status= 'watching' or status = 'finish' ),  
    user_id int,  
    movie_id int,
```

```
series_id int,

FOREIGN KEY (movie_id) references Movies(id),
FOREIGN KEY (series_id) references Series(id),
FOREIGN KEY (user_id) references Users(id)
);

-- Add 5 Users
INSERT INTO Users (username, password, email, subscribe) VALUES
('shefo', '123456', 'shefo@gmail.com', true),
('majd', '123456', 'majd@gmail.com', true),
('renad', '123456', 'renad@gmail.com', false),
('turki', '123456', 'turki@gmail.com', true),
('faisal', '123456', 'faisal@gmail.com', false);

-- Shefo Favorite List
INSERT INTO FivList (status, user_id, movie_id, series_id) VALUES
('finish', 1, 1, NULL),
('watching', 1, NULL, 1),
('new', 1, 3, NULL);

-- Majd Favorite List
INSERT INTO FivList (status, user_id, movie_id, series_id) VALUES
('watching', 2, 2, NULL),
('finish', 2, NULL, 2),
('new', 2, NULL, 3);

-- Renad Favorite List
INSERT INTO FivList (status, user_id, movie_id, series_id) VALUES
('new', 3, 4, NULL),
('watching', 3, NULL, 4),
('finish', 3, 5, NULL);

-- Turki Favorite List
INSERT INTO FivList (status, user_id, movie_id, series_id) VALUES
('finish', 4, 3, NULL),
('new', 4, NULL, 5),
('watching', 4, 1, NULL);

-- Faisal Favorite List
INSERT INTO FivList (status, user_id, movie_id, series_id) VALUES
```

```
('watching', 5, NULL, 1),  
('finish', 5, 2, NULL),  
('new', 5, NULL, 3);
```

-- Shefo History

```
INSERT INTO History (status, user_id, movie_id, series_id) VALUES  
('finish', 1, 1, NULL),  
('watching', 1, NULL, 1),  
('finish', 1, 2, NULL);
```

-- Majd History

```
INSERT INTO History (status, user_id, movie_id, series_id) VALUES  
('finish', 2, NULL, 2),  
('watching', 2, 3, NULL),  
('finish', 2, 4, NULL);
```

-- Renad History

```
INSERT INTO History (status, user_id, movie_id, series_id) VALUES  
('watching', 3, NULL, 3),  
('finish', 3, 5, NULL),  
('finish', 3, NULL, 4);
```

-- Turki History

```
INSERT INTO History (status, user_id, movie_id, series_id) VALUES  
('finish', 4, 1, NULL),  
('watching', 4, NULL, 5),  
('finish', 4, 3, NULL);
```

-- Faisal History

```
INSERT INTO History (status, user_id, movie_id, series_id) VALUES  
('watching', 5, 2, NULL),  
('finish', 5, NULL, 1),  
('finish', 5, 4, NULL);
```

## 6. Sample Data

The database contains sample data to demonstrate how the system works.

It includes 5 movies, 5 series, and 5 users. Each user also has sample records in their favorite list and viewing history.

The sample data can be used to test relationships between the tables and perform SQL queries.

## 7. Conclusion

The Pixel Database System demonstrates the basic structure of a streaming platform using a relational database. It organizes users, movies, series, favorite lists, and viewing history while connecting the data through foreign keys.

The project demonstrates the use of database concepts such as table creation, primary keys, foreign keys, constraints, relationships, and data insertion.